

Multi-agent Framework for Time-Sensitive Complementary Collaboration in Minecraft

Juheon Yi, Jinglu Wang, Xiaoyi Zhang, Yan Lu

Microsoft Research Asia

{jyi,jinglwa,xiaoyizhang,yanlu}@microsoft.com

Abstract

We present **TickingCollabBench**, a Minecraft-based multi-agent benchmark for a novel class of *time-sensitive complementary collaboration tasks*. Our benchmark reflects four core characteristics of real-world collaboration: agent heterogeneity, mandatory collaboration, dynamic environments, and strict real-time constraints with failure risks. To enable this, we develop the **TickingCollab** framework, which supports the generation of diverse dynamic events and abstracts Minecraft’s primitive APIs to enable declarative **YAML task specifications for composing these events**. Building on this, we design a feasibility-aware automated benchmark generation pipeline, where an LLM drafts structurally diverse task configurations and feasibility verifier filters out invalid ones using approximate constraints. Evaluations demonstrate that long latency and the inherent difficulty of coordinating under partial observability and agent heterogeneity cause LLMs to frequently fail under dynamic environments and fall significantly short of a global-knowledge oracle.

1 Introduction

Real-world multi-agent collaboration often requires agents with partial observability to synergistically combine heterogeneous capabilities and complete tasks under strict time constraints. For instance, a team of embodied robots with different tools and mobility may need to coordinate to respond to spreading hazards before a rescue deadline. Similarly, in collaborative work settings, personal agents running on different users’ devices may observe only local data and have heterogeneous computing resources, requiring them to jointly process distributed information to respond to users’ requests in a timely manner. However, composing such time-sensitive collaborative scenarios in the real world and evaluating agents at scale is difficult due to safety risks, deployment costs, and limited controllability over environment dynamics. As a result, many prior works focus on static tasks with shared context, homogeneous agents, and no explicit time-to-failure constraints [27, 4, 22].

To bridge this gap, Minecraft has emerged as a scalable testbed for composing complex tasks and dynamic environments, and systematically controlling agent capabilities such as tools, mobility, and perception. However, existing Minecraft-based multi-agent collaboration benchmarks [20, 18, 13, 23, 5] still suffer from critical limitations:

- **Insufficient emphasis on real-time, dynamic collaboration.** Existing tasks often feature homogeneous agents and are individually solvable, rendering genuine collaboration optional rather than mandatory. Furthermore, their static environments and best-effort objectives allow agents to rely on one-time offline planning, facing little to no failure risk from delayed decisions or runtime dynamic events (quantitative analysis in Table 2).
- **Limited framework support for dynamic tasks.** Existing Minecraft frameworks are primarily designed for static environments and lack built-in support for runtime dynamic

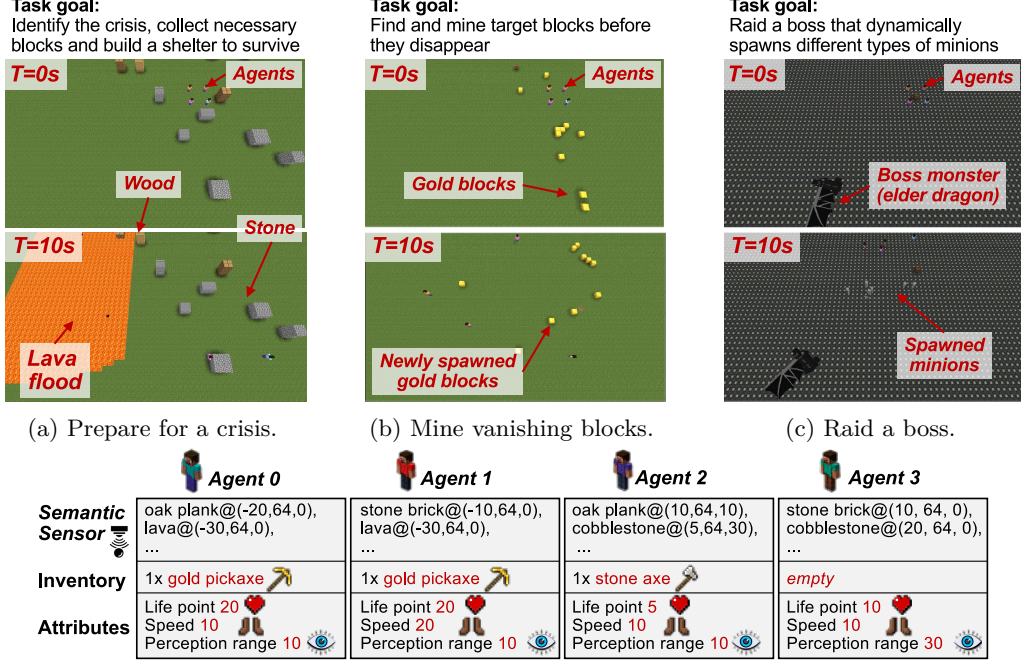


Figure 1: Time-sensitive complementary collaboration tasks in **TickingCollabBench**.

events (e.g., spreading floods, object/monster spawn and despawn). Consequently, to introduce such dynamics, agent developers must utilize low-level Minecraft APIs to build custom server plugins from scratch. This high technical barrier creates severe development overhead, thereby limiting the creation of diverse and complex collaboration scenarios.

We present **TickingCollab**, a benchmark suite and framework for evaluating LLM agents on a novel class of *time-sensitive complementary collaboration tasks* in Minecraft. **TickingCollabBench** (Figure 1) targets scenarios where agents with heterogeneous capabilities and partial observability must tightly integrate their complementary skills. Crucially, the environments continuously change, and failure to rapidly adapt directly causes task failure. Supported by quantitative comparisons with prior benchmarks (Tables 1 and 2), we pose a fundamental question: *can LLMs orchestrate accurate and efficient collaboration across heterogeneous agents, when faced with dynamic environments and real-time failure risks?*

To systematically construct and evaluate time-sensitive complementary collaboration tasks, our **TickingCollab** framework provides three key functionalities:

- **Dynamic environment manager.** Developers can declaratively inject complex runtime dynamics (e.g., lava waves, object/monster spawn/despawn) via Minecraft API-free YAML configurations (Listing 1), bypassing the severe overhead of custom plugin development.
- **Feasibility-aware automated benchmark generation.** To systematically explore a large and complex parameter space in composing time-sensitive complementary collaboration tasks, we design an automated pipeline where an LLM drafts diverse task configurations and a feasibility verifier filters out invalid ones via approximate constraints.
- **Comprehensive evaluation.** The framework isolates LLM’s planning accuracy from inference latency via dual execution modes (*synchronous fixed-timestep* vs. *asynchronous real-time*), while supporting parallel simulation and fine-grained system cost logging.

We evaluate our benchmark using a baseline multi-agent collaboration scheme (**TickingCollabAgent**) with two distinct coordination policies (centralized and distributed) motivated from prior works [13, 20]. Our evaluation reveals that LLM inference latency poses a critical bottleneck in real-time asynchronous execution, frequently causing task failures due to time constraint violations. Furthermore, while centralized coordination outperforms distributed topologies by mitigating communication and inference overheads, it still underperforms an oracle—a non-LLM solution that leverages global, ground-truth access to the dynamic environment and human-crafted scheduling rules. These findings underscore the

Table 1: Comparison of prior Minecraft agent benchmarks and **TickingCollabBench**.
 Δ : partially covered for the subset of tasks listed in parentheses.

	Benchmark	Heterogeneous agent capabilities?	Mandatory collaboration?	Dynamic environment?	Real-time constraints (or failure risks)?
Single agent	MineRL [9]	×	×	×	×
	MineDojo [6]	×	×	Δ (combat)	Δ (combat, survive)
	Odyssey [12]	×	×	Δ (combat)	Δ (combat, survive)
	MCU [25]	×	×	Δ (combat)	Δ (combat, survive)
Multi-agent	MineLand [23]	×	×	Δ (combat)	Δ (combat, survive)
	TeamCraft [13]	○ (different items)	○	×	×
	MineCollab [20]	○ (different items)	Δ (cook, craft)	×	×
	PillagerBench [18]	×	×	○	○ (vs. opponents)
	VillagerBench [5]	×	Δ (escape room)	Δ (harvest)	×
	TickingCollabBench	○	○	○	○

challenges of heterogeneous multi-agent planning under partial observability and necessitate efficient LLM inference and multi-agent coordination policies in dynamic environments.

2 TickingCollabBench Benchmark Suite

TickingCollabBench consists of three representative *time-sensitive complementary collaboration tasks* (Figure 1), where agents with heterogeneous capabilities must coordinate to achieve a global objective under partial observability of the dynamic environment. These tasks are designed as controlled analogues of the real-world collaboration scenarios discussed in Section 1, emphasizing four key properties that are underrepresented in previous Minecraft multi-agent benchmarks (see Table 1 for a detailed comparison):

- **Heterogeneous capabilities.** Agents differ in action-relevant attributes and resources (e.g., perception range, speed, health, tools), necessitating complex complementary roles that cannot be captured by simple inventory differences featured in prior benchmarks.
- **Mandatory collaboration.** Tasks are constructed so that success requires coordinating complementary capabilities, rather than simply scaling identical agents.
- **Dynamic environments.** Environments continuously change at runtime, invalidating one-shot offline plans and requiring online adaptation.
- **Real-time constraints.** Unlike prior penalty-free settings, delayed decision-making directly causes task failure, demanding timely execution.

TickingCollab framework provides a declarative interface for specifying these tasks. This enables an automated benchmark generation pipeline: an LLM first generates diverse configurations incorporating the four key properties, which are then filtered based on feasibility criteria to yield 634 valid tasks for **TickingCollabBench** (details in Section 3.1).

2.1 Task Suite

Task #1: Prepare for a crisis (Figure 1a). Agents must identify an approaching crisis (e.g., lava flood, avalanche) and collaboratively gather appropriate materials scattered in the map to build an appropriate survival shelter (e.g., stone instead of flammable wood) before crisis impact. Agents possess different mining tools (e.g., axes for woods, pickaxes for metal ores), perception ranges, and movement speeds. Survival depends on efficient role allocation, such as utilizing long-perception agents as scouts while faster agents collect distant blocks. Unlike prior *construction tasks* that ignore time constraints and failure risks, this task demands timely execution and efficient coordination of heterogeneous agents.

Task #2: Mine vanishing blocks (Figure 1b). Agents must mine target quotas for multiple block types that randomly appear and vanish after type-specific lifetimes. Given distinct movement speeds, perception ranges, and heterogeneous mining tools that dictate both block compatibility and mining efficiency (e.g., wood requires an axe, gold ore requires at least an iron-tier pickaxe, and higher-tier tools like diamond yield faster mining rates), agents must optimally assign targets by calculating travel and mining times against block lifetimes to avoid wasted effort. While previous *harvesting tasks* mostly feature static block placements and uniform agents, our task requires dynamic, capability-aware assignment.

Table 2: Comparison of multi-agent benchmark statistics. $\uparrow$ and $\downarrow$ indicate whether lower or higher values imply a more challenging task, respectively.

	MineLand [23]			TeamCraft [13]				MineCollab [20]			TickingCollabBench		
	Combat	Harvest	Build	Break	Build	Farm	Smelt	Build	Cook	Craft	Prepare	Mine	Raid
$\mathcal{H} \uparrow$	0	0	0	0.72	0.43	0.39	0.68	0	0.43	0.44	0.78	0.72	0.31
$\mathcal{N} \uparrow$	0	0	0	0	0.54	1.13	0	0	1.87	1.76	1.11	1.42	1.39
$\mathcal{D} \uparrow$	0	0	0	0	0	0	0	0	0	0	1.79	2.98	0.38
$\tau \downarrow$	11.51s*	∞	∞	∞	∞	∞	∞	∞	∞	∞	44.03s	33.31s	25.37s

* Samples with $\tau = \infty$ are excluded.

Task #3: Raid a boss (Figure 1c). Agents must defeat a boss monster that dynamically spawns various minions with different life points (HP) and damages. Agents differ in base life points and weapons with type-specific damage multipliers. Agents must jointly optimize target assignments based on type advantages and survivability, while strategically disengaging to consume health potions at scattered chests. Whereas prior *combat tasks* typically assume statically generated monsters and homogeneous agents, ours introduces dynamic enemy spawns and demands intricate combat coordination among diverse roles.

2.2 Collaboration Difficulty Metrics

We define four collaboration difficulty metrics to quantitatively evaluate how **TickingCollabBench** reflects time-sensitive, complementary collaboration.

- **Agent heterogeneity ($\mathcal{H}$)** measures the average pairwise normalized distance of attributes across all unique agent pairs P :

$$\mathcal{H} = \frac{1}{|P|} \sum_{(i,j) \in P} \left(\frac{1}{|K|} \sum_{k \in K} \delta_k(a_i, a_j) \right).$$

For each attribute $k \in K$, the distance δ_k handles continuous values (e.g., HP) via min-max normalization and sets (e.g., inventory items) via Jaccard distance:

$$\delta_k(a_i, a_j) = \begin{cases} \frac{|v_{ik} - v_{jk}|}{v_k^{max} - v_k^{min}} & \text{if } k \text{ is continuous} \\ 1 - \frac{|S_{ik} \cap S_{jk}|}{|S_{ik} \cup S_{jk}|} & \text{if } k \text{ is a set} \end{cases}$$

where v_k^{max} and v_k^{min} denote predefined bounds of the parameter space. Thus, $\mathcal{H} \in [0, 1]$, where 1 implies maximal distinctiveness.

- **Collaboration necessity ($\mathcal{N}$)** estimates the ratio of the total task workload to the maximum single-agent capacity:

$$\mathcal{N} = \min_{a \in A} \left(\sum_k \frac{\text{Workload}_k}{\text{Throughput}_{a,k}} \right) / T_{max}.$$

The inner sum computes the time required for the most capable single agent $a \in A$ to sequentially process all targets k . Here, Workload_k is the required block count (Tasks #1, #2) or total enemy HP (Task #3), while $\text{Throughput}_{a,k}$ is agent a 's mining speed or damage-per-second (DPS). Note that $\mathcal{N}$ is a conservative lower bound, as the time required for agent planning and movement are omitted; thus, $\mathcal{N} > 1$ strictly guarantees that collaboration is mandatory.

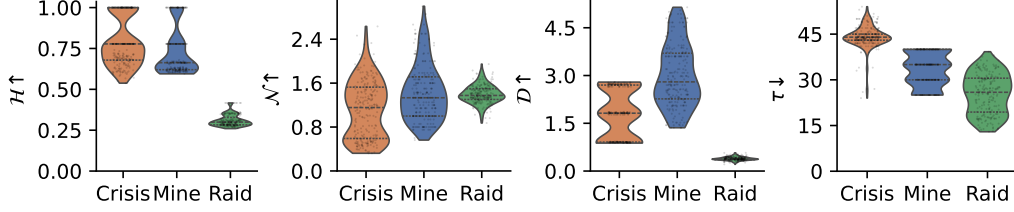
- **Environment dynamicity ($\mathcal{D}$)** is quantified as:

$$\mathcal{D} = (\text{Total Environment State Changes}) / T_{max}.$$

A "state change" is any environment modification independent of agent actions, such as entity spawns/despawns (Tasks #2, #3) or crisis spread (Task #1). Higher $\mathcal{D}$ necessitates continuous online replanning.

- **Time-to-failure (τ)** measures the time window before irreversible failure occurs, which is defined as: crisis arrival time (Task #1), minimum block lifespan (Task #2), or the time for spawned enemies to defeat all agents, calculated as $\frac{\text{Total Agent HP}}{\text{Total Enemy DPS}}$ averaged across spawn events (Task #3). Lower τ demands faster decision-making.

As shown in Table 2, prior Minecraft benchmarks typically feature static environments, negligible failure risks, and limited agent heterogeneity (differing only in inventories), rarely necessitating true collaboration outside a few exceptions (e.g., crafting or cooking in MineCollab [20], combat in MineLand [23]). In contrast, **TickingCollabBench** yields significantly higher metric scores, indicating far more challenging collaboration tasks. Figure 2 and Table 3 further show that the generated configurations cover broad ranges of difficulty and task



(a) Heterogeneity ($\mathcal{H}\uparrow$). (b) Necessity ($\mathcal{N}\uparrow$). (c) Dynamicity ($\mathcal{D}\uparrow$). (d) Time-to-failure ($\tau\downarrow$).

Figure 2: Distribution of collaboration difficulty metrics in **TickingCollabBench**. Internal lines show quartiles (Q1, median, Q3); overlaid dots show individual configurations.

Table 3: Parameter distributions (min / mean $\pm$ std / max for continuous values).

Prepare for a crisis		Mine vanishing blocks		Raid a boss	
# Agents	2 / 3.88 $\pm$ 1.90 / 8	# Agents	2 / 5.12 $\pm$ 2.25 / 8	# Agents	3 / 5.93 $\pm$ 1.54 / 8
Crisis type	lava 30%, snow 32%, water 38%	Tool tier	golden 14%, iron 31%, stone 32%, wooden 23%	Boss HP	210 / 241.53 $\pm$ 18.73 / 280
Crisis speed	1 / 1.98 $\pm$ 0.82 / 3	# Target types	2: 33%, 3: 36%, 4: 32%	Minion HP	25 / 34.60 $\pm$ 3.87 / 40
Agent speed	3 / 4.49 $\pm$ 1.09 / 6	Block lifetime	25 / 33.30 $\pm$ 5.36 / 40	# Minions/wave	2 / 2.50 $\pm$ 0.56 / 4
		# Blocks/wave	8 / 9.03 $\pm$ 0.70 / 10	Wave interval	8 / 11.34 $\pm$ 1.88 / 16

parameters. Across the three tasks, these distributions reflect diverse collaboration challenges that vary in their focus (e.g., strict time-to-failure constraints in “Raid a boss” task, and agent heterogeneity in the other two).

3 TickingCollab Framework

Figure 3 illustrates the **TickingCollab** framework architecture.

- **Task metadata generator** (Section 3.1) provides descriptive task composition via declarative YAML without requiring Minecraft API expertise, enabling an automated generation pipeline to construct **TickingCollabBench**.
- **Task orchestrator** (Section 3.2) translates the metadata into dynamic runtime events via the *dynamic environment manager* and bridges agents to the Minecraft server using Mineflayer [15]. It supports both synchronous (fixed-timestep) and asynchronous (real-time) execution to decouple the LLM agent’s reasoning accuracy from latency.
- **Multi-agent runtime** (Section 3.3) provides a modular abstraction for the *agent core* and *communication manager*, streamlining the development of custom collaborative agents.

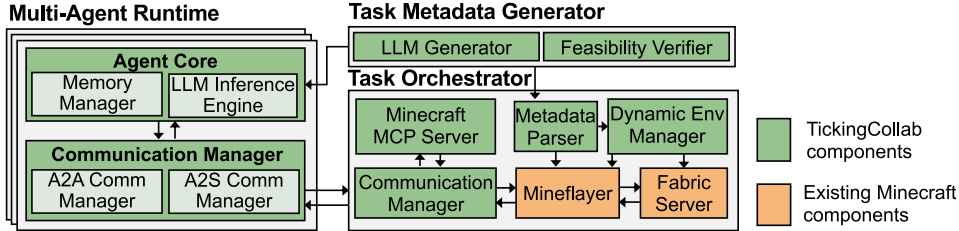


Figure 3: **TickingCollab** framework architecture.

3.1 Task Metadata Generator and Automated Benchmark Generation

Listing 1 illustrates the task metadata structure, comprising four key fields: (i) *task description*, (ii) *environment layout*, (iii) *events* (with dynamic triggers and patterns), and (iv) *agents*. While our declarative interface abstracts away complex Minecraft API programming, manual task design still remains challenging due to the massive parameter space (e.g., jointly aligning crisis type and speed, necessary survival block placements, and agents’ speed, perception range, and mining tools). Prior Minecraft multi-agent benchmarks typically perform naive parameter sweeping (e.g., merely adjusting N identical agents vs. M skeletons) under static environments, yielding trivial variations that fail to meaningfully emphasize collaboration.

Listing 1: Example metadata (“prepare for a crisis”, full list in Supp. Section A).

```

1 task:
2   - goal: "Identify the origin and type of the crisis, and gather necessary blocks to
   build a shelter for survival."
3 environment:
4   - {type: cobblestone, position: [-5,64,15], num_blocks: 10},
5   - {type: oak_log, position: [-8,64,10], num_blocks: 8},
6   ...
7 events:
8   {id: lava_wave, trigger: {start: 5, end: 100}, actions: {type: progressive_fill,
   block: lava, area: {min: [-40,64,-20], max: [40,65,20]}, direction: east, speed: 2}
9 agents:
10  {name: MineplayerBot0, position: [8,64,18], inventory: {gold_pickaxe: 1},
11   capabilities: {perception range: 10; speed: 10},
   ...

```

To overcome this challenge, we design a *feasibility-aware automated generation pipeline* to construct **TickingCollabBench** at scale. First, users define task goals and parameter spaces via a task metadata template (example in Supp. Section B). Using this, the LLM navigates the parameter space to draft structurally diverse configurations spanning various environments, agent compositions, and difficulty levels. The LLM may occasionally generate unsolvable task configurations (e.g., requiring gold ore mining without pickaxes, or setting enemy HP beyond the agents’ maximum damage output); thus, a *feasibility verifier* screens out invalid configurations using constraints that model the task feasibility.

Table 4 details the task parameter space (designed to reflect the four properties in Table 1) and the feasibility verification criteria. Even setting aside runtime stochasticity (e.g., random spawn positions and agent trajectories), optimal target assignment and sequencing (e.g., which blocks to mine) remain intractable. With heterogeneous agent capabilities and strict time constraints, this reduces to an NP-hard routing and allocation problem (analogous to the multi-agent traveling salesman problem), precluding exact guarantees. We thus introduce margins α, β, γ to define approximate feasibility constraints that also control task difficulty. Generating 250 configurations per task with GPT-5.1 and filtering them with margin values 2.0 yielded 634 valid configurations (225, 219, and 190 for Tasks #1, #2, and #3); see Supp. Section D for margin sensitivity analysis and Section E for generated examples.

Table 4: **TickingCollab**’s automated benchmark generation: parameter space and feasibility verification criteria. Details on each variable in verification criteria are in Supp.-Section C.

Parameter space	Verification criteria
Task #1: Prepare for a crisis	
- Crisis config: $\mathcal{C} = (Type, P, V, H)$ – Type: crisis type (e.g., lava, water, snow); P, V, H : origin, speed, height.	- Necessary mining items? $\max_n(\tau_{E_n}) \geq \tau_{req}(b), \forall b \in \mathcal{B}_{target}$
- Agent config: $\mathcal{A}_n = (V_n, R_n, E_n)$ – V_n, R_n : movement speed and perception range; E_n : mining tool with tier τ_{E_n} . Block type i is mineable only if $\tau_{E_n} \geq \tau_{req}(i)$, and takes time $T_{mine}^{n,i}$.	- Sufficient survival blocks? $N_{required} \geq (H + 1) \cdot N_{agent}$
- Block placements: $\mathcal{I} = \{(Type_j, Loc_j)\}$ – types and locations.	- Sufficient preparation time? $T_{crisis} \geq \alpha \cdot (T_{gather} + T_{construct})$ (α : time margin)
- Simulation duration: T_{max} steps.	
Task #2: Mine vanishing blocks	
- Target blocks: $\mathcal{B}_i = (Type_i, N_{goal}^i)$ – type and count.	- Necessary mining items? $\max_n(\tau_{E_n}) \geq \tau_{req}(b), \forall b \in \mathcal{B}_{target}$
- Agent config: $\mathcal{A}_n = (V_n, R_n, E_n)$ – same as Task #1.	- Sufficient block spawns? $N_{spawn}^i \geq N_{goal}^i, \forall i$
- Spawn pattern: $\mathcal{S}_i = (T_{start}^i, T_{end}^i, T_{int}^i, T_{life}^i)$ – start, end, interval, lifetime.	- Sufficient block lifetimes? $\beta \cdot (\min_n(T_{move} + T_{mine}^{n,i})) \leq T_{life}^i, \forall i$ (β : time margin)
- Simulation duration: T_{max} steps.	
Task #3: Raid a boss	
- Boss config: $\mathcal{B} = (HP_B, D_B)$ – boss life point and damage.	- Sufficient agent damage? $\gamma \cdot T_{max} \cdot \sum_n D_n \geq HP_B + \sum_i (N_i \cdot HP_M^i)$ (γ : efficiency margin)
- Minion spawns: $\mathcal{P}_i = (T_{start}^i, T_{end}^i, T_{int}^i, N^i, HP_M^i, D_M^i)$ – start, end, interval of i -th minion spawn events; N^i : minion count per spawn; HP_M^i, D_M^i : minion life point, damage.	
- Agent config: $\mathcal{A}_n = (HP_n, D_n)$ – life point and damage.	
- Simulation duration: T_{max} steps.	

3.2 Task Orchestrator

Component #1: Dynamic environment manager. We build a plugin abstracting Minecraft’s complex primitive APIs (e.g., `/effect`, `/attribute`) to automatically compose and manage dynamic environment specified in metadata. Unlike prior benchmarks reliant on static environments, this manager handles diverse, complex runtime dynamics. It is also highly extensible, allowing developers to inject custom interaction mechanisms not natively supported by Minecraft (e.g., an inventory weight system, region-specific debuffs).

Component #2: Communication manager. This module bridges the agent runtime and Mineflayer-controlled bots [15] connected to Minecraft server. It maps high-level agent actions to predefined JavaScript code executed on the Fabric server (Supp. Section F) and returns structured observations (agent status and sensor data) for the next decision step.

Component #3: Multi-agent collaboration evaluator. To enable in-depth analysis of the agent’s reasoning capabilities and system-level costs (Section 4), our framework supports parallel simulation execution, comprehensive logging (e.g., number of inferences and inference latency, token usage, communication overhead), and two distinct evaluation modes:

- **Synchronous (fixed-timestep) mode** pauses the simulation during LLM inference. It isolates and evaluates pure decision-making accuracy by ignoring inference latency.
- **Asynchronous (real-time) mode** runs the simulation continuously. It evaluates both accuracy and latency, as prolonged LLM inference causes actions to become stale against the rapidly changing environment, directly impacting task success.

3.3 Multi-agent Runtime and TickingCollabAgent

Our modular runtime decouples the agent core (LLM/memory) from communication modules, enabling flexible logic customization. We implement a baseline **TickingCollabAgent** with two distinct coordination policies (system prompts and toolsets in Supp. Sections F and G):

- **TickingCollabAgent-centralized** (motivated by TeamCraft [13]). A designated *master* agent aggregates all peer agents’ observations, performs joint planning, and dispatches actions. While centralizing global observations maximizes joint planning quality, it introduces synchronization bottlenecks; the master must wait for all agents to report back before replanning, leaving faster agents idle due to varying action latencies.
- **TickingCollabAgent-distributed** (motivated by MineCollab [20]). Agents plan independently in parallel, resolving shared decisions (e.g., role allocation) via a propose–wait–act negotiation protocol. We relax MineCollab’s 1-to-1 messaging restriction to support selective multi-agent broadcasting, facilitating efficient coordination under time constraints.

4 Experiments

We host two LLMs (GPT-5.1 and DeepSeek-R1 [8]) in Azure AI Foundry as the backbone of **TickingCollabAgent**, and evaluate their task success rates on our benchmark. We use an Ubuntu 22.04.5 machine with an AMD EPYC 9V84 CPU (80 logical CPUs), 629 GiB RAM. We implement our **TickingCollab** framework on top of Minecraft Java Edition 1.19, using a Fabric server (loader v0.14.18) and Mineflayer 4.14.0 (unofficial third-party modifications to Minecraft) as the bot control interface.

Table 5: Average task success rate of **TickingCollabAgent**.

Task	GPT-5.1				DeepSeek-R1				Oracle
	Centralized		Distributed		Centralized		Distributed		
	Sync	Async	Sync	Async	Sync	Async	Sync	Async	
Prepare for a crisis	0.42	0.15	0.24	0.02	0.26	0.02	0.03	0.01	0.91
Mine vanishing blocks	0.62	0.05	0.54	0.00	0.65	0.00	0.35	0.01	0.80
Raid a boss	0.28	0.06	0.40	0.03	0.37	0.01	0.29	0.02	0.59

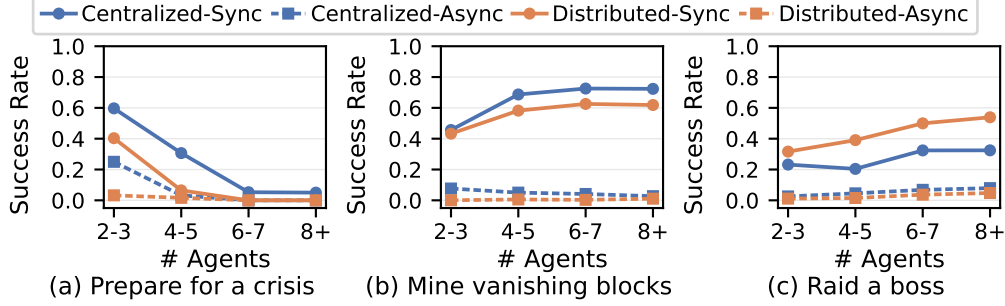


Figure 4: Task success rate across different numbers of agents.

4.1 Overall Task Success Rate

Table 5 shows the overall task success rates across various tasks, LLMs, and execution modes. To evaluate the LLM’s planning accuracy, we designed an oracle solution where a centralized agent has access to ground-truth task metadata and runtime events:

- **Task #1 Prepare for a crisis:** the oracle retrieves the crisis type, origin, and speed from the task metadata at the beginning of the simulation.
- **Task #2 Mine vanishing blocks & Task #3 Raid a boss:** the oracle receives real-time object spawn/despawn events from task orchestrator (Section 3.2) during runtime.

Based on this information, the oracle plans the actions of individual agents. Since scheduling heterogeneous agents is an NP-hard problem (analogous to the multi-agent traveling salesman problem), we designed an efficient, non-LLM-based task allocation algorithm using handcrafted heuristics. Note that scheduling in Task #2 is more challenging than in Task #1 due to a greater variety and volume of target blocks. Furthermore, moving enemies and agent deaths from attacks in Task #3 make planning non-trivial, even for the oracle.

Overall, we observe the following trends. First, most tasks fail in async mode, as the average 20-second API delay frequently exceeds our benchmark’s time-to-failure constraints (Figure 2(d)). Second, in sync mode, **TickingCollabAgent-centralized** generally outperforms **distributed**. In distributed topology, inter-agent communication and inference overheads result in most of the time budget being spent for planning, leaving insufficient time for actions (detailed in Figures 5 and 6). Finally, even in sync mode, **centralized** falls short of the oracle’s success rate. This highlights the difficulty of planning multi-agent collaboration in dynamic environments using partial observations rather than global ground-truth knowledge. These highlight the necessity of an efficient multi-agent coordination policy coupled with fast and accurate LLM planning. Refer to Supp. Section H for operational timeline examples.

4.2 Ablation Study

We analyze the factors behind **TickingCollabAgent**’s low success rate. Given consistent trends across both evaluated LLMs, we present results for GPT-5.1.

Scaling with team size. Figure 4 shows the task success rate across different numbers of agents. For “raid a boss” and “mine vanishing blocks” tasks, increasing the number of agents improves enemy-killing or mining throughput, leading to higher success rates. In contrast, in “prepare for a crisis”, increasing the number of agents increases both the required survival blocks to mine and the shelter size. Since the success condition requires all agents to survive, the success rate tends to decrease. Across all tasks, asynchronous mode generally yields lower success rates due to LLM inference latency (typically around 20 seconds per API call).

System costs. Figure 5 reports the communication and inference costs of the distributed baseline in synchronous mode. Because agents can broadcast messages to multiple peers, message volume and LLM inference calls increase rapidly as team size increases (Figures 5(a) and (b)). Consequently, token usage and wall-clock time spike, often approaching the 40-minute simulation timeout (Figures 5(c) and (d)). This highlights the need for efficient communication protocols and group formation strategies in distributed multi-agent systems.

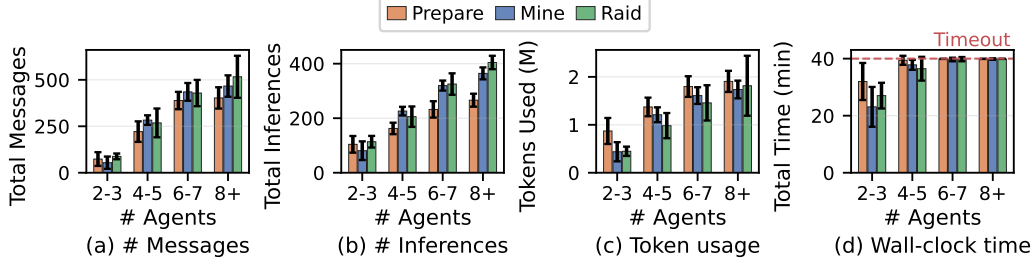


Figure 5: System costs of **TickingCollabAgent-distributed** in synchronous mode.

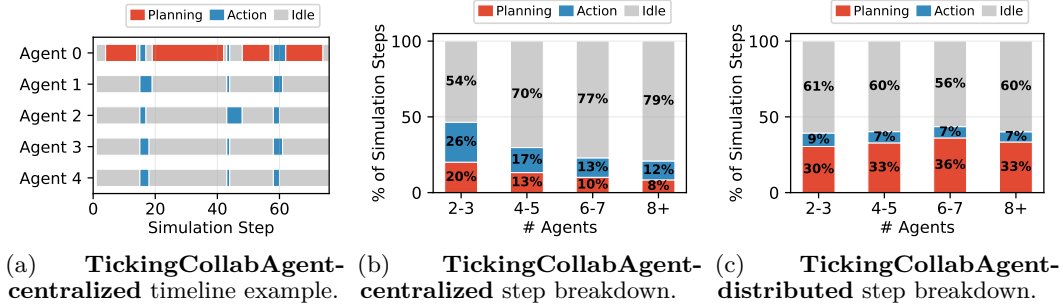


Figure 6: Step-level planning, action, and idle-time breakdown in asynchronous mode.

Coordination overhead. Figure 6 breaks down asynchronous mode execution into planning, action, and idle steps. In the centralized baseline, agents remain idle while the central planner waits for LLM inference (typically around 20 seconds per API call); additional idle time arises as agents’ assigned actions vary in duration (Figure 6(a)). This synchronization bottleneck becomes more pronounced as team size grows, leading to a larger idle fraction (Figure 6(b)). The distributed baseline avoids a single central planner, but spends more time in planning because agents must communicate and negotiate before acting (Figure 6(c)). Overall, these results highlight the need for agents to form appropriate coordination topologies and optimize both planning and action execution times.

5 Limitations and Future Work

Observation modality. We use distance-limited structured semantic sensors to focus on the collaborative planning capabilities of LLM agents. We plan to extend **TickingCollabBench** to multimodal inputs (e.g., first-person-view video and audio) to evaluate VLM agents.

Agent-environment dynamics. **TickingCollab** supports complex agent-conditioned interactions (e.g., inventory-based movement slowdowns). We will leverage these primitives to test collaboration under complex, cascading environmental changes driven by agent actions.

6 Related Work

Minecraft agents. While numerous works leverage Minecraft to develop autonomous agents [19, 1, 14, 10, 2, 7, 6], most focus on single-agent scenarios. Recent multi-agent extensions [13, 23, 20, 5, 18] mostly feature static environments, homogeneous agents, and tasks where collaboration is optional or lacks real-time constraints. Bridging this gap from real-world collaboration, **TickingCollabBench** introduces time-sensitive, complementary tasks requiring strictly mandatory coordination.

Multi-agent collaboration. Broader evaluations of multi-agent collaboration in domains like mathematics, science, or coding [27, 4, 22] often lack environmental dynamics and explicit time limits. **TickingCollabBench** strictly demands both decision accuracy and low latency. Other orthogonal efforts improving multi-agent efficiency—such as task plan search [28, 24], communication topologies [26, 11, 17, 27], and resource-aware planning [21, 3]—can be seamlessly integrated to tackle the real-time challenges in our benchmark.

7 Conclusion

We presented **TickingCollabBench**, a Minecraft benchmark evaluating *time-sensitive complementary collaboration*. To capture the core characteristics of real-world collaboration (agent heterogeneity, mandatory coordination, dynamic environments, and real-time constraints), we developed an extensible orchestration framework and an automated task generation pipeline. Baseline evaluations reveal significant performance gaps in current LLMs, highlighting the critical need for more accurate and latency-efficient multi-agent systems.

Acknowledgement. This study is conducted for research only, not for an actual Minecraft product.

References

- [1] A. Bolton, A. Lerchner, A. Cordell, A. Moufarek, A. Bolt, A. Lampinen, A. Mitenkova, A. O. Hallingstad, B. Vujatovic, B. Li, et al. Sima 2: A generalist embodied agent for virtual worlds. *arXiv preprint arXiv:2512.04797*, 2025.
- [2] S. Cai, Z. Mu, K. He, B. Zhang, X. Zheng, A. Liu, and Y. Liang. Minestudio: A streamlined package for minecraft ai agent development. *arXiv preprint arXiv:2412.18293*, 2024.
- [3] S. Cai, Y. Ning, and H. Liu. Agentbalance: Backbone-then-topology design for cost-effective multi-agent systems under budget constraints. *arXiv preprint arXiv:2512.11426*, 2025.
- [4] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *ICLR*, 2024.
- [5] Y. Dong, X. Zhu, Z. Pan, L. Zhu, and Y. Yang. Villageragent: A graph-based multi-agent framework for coordinating complex task dependencies in minecraft. *arXiv preprint arXiv:2406.05720*, 2024.
- [6] L. Fan, G. Wang, Y. Jiang, A. Mandlekar, Y. Yang, H. Zhu, A. Tang, D.-A. Huang, Y. Zhu, and A. Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge. *Advances in Neural Information Processing Systems*, 35:18343–18362, 2022.
- [7] R. Gong, Q. Huang, X. Ma, Y. Noda, Z. Durante, Z. Zheng, D. Terzopoulos, L. Fei-Fei, J. Gao, and H. Vo. Mindagent: Emergent gaming interaction. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 3154–3183, 2024.
- [8] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, et al. Deepseek-rl: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [9] W. H. Guss, B. Houghton, N. Topin, P. Wang, C. Codel, M. Veloso, and R. Salakhutdinov. Minerl: A large-scale dataset of minecraft demonstrations. *arXiv preprint arXiv:1907.13440*, 2019.
- [10] M. Li, Z. Wang, K. He, X. Ma, and Y. Liang. Jarvis-vla: Post-training large-scale vision language models to play visual games with keyboards and mouse. *arXiv preprint arXiv:2503.16365*, 2025.
- [11] X. Li, X. Wang, C. Bai, and J. Zhang. Exponential topology-enabled scalable communication in multi-agent reinforcement learning. *arXiv preprint arXiv:2502.19717*, 2025.
- [12] S. Liu, Y. Li, K. Zhang, Z. Cui, W. Fang, Y. Zheng, T. Zheng, and M. Song. Odyssey: Empowering minecraft agents with open-world skills. *arXiv preprint arXiv:2407.15325*, 2024.
- [13] Q. Long, Z. Li, R. Gong, Y. N. Wu, D. Terzopoulos, and X. Gao. Teamcraft: A benchmark for multi-modal multi-agent systems in minecraft. *arXiv preprint arXiv:2412.05255*, 2024.
- [14] L. Magne, A. Awadalla, G. Wang, Y. Xu, J. Belofsky, F. Hu, J. Kim, L. Schmidt, G. Gkioxari, J. Kautz, et al. Nitrogen: An open foundation model for generalist gaming agents. *arXiv preprint arXiv:2601.02427*, 2026.
- [15] PrismarineJS. Mineflayer. <https://github.com/PrismarineJS/mineflayer>, 2014.
- [16] PrismarineJS. Mineflayer-pvp. <https://github.com/PrismarineJS/mineflayer-pvp>, 2025.

- [17] C. Qian, Z. Xie, Y. Wang, W. Liu, K. Zhu, H. Xia, Y. Dang, Z. Du, W. Chen, C. Yang, et al. Scaling large language model-based multi-agent collaboration. *arXiv preprint arXiv:2406.07155*, 2024.
- [18] O. Schipper, Y. Zhang, Y. Du, M. Pechenizkiy, and M. Fang. Pillagerbench: Benchmarking llm-based agents in competitive minecraft team environments. In *2025 IEEE Conference on Games (CoG)*, pages 1–15. IEEE, 2025.
- [19] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [20] I. White, K. Nottingham, A. Maniar, M. Robinson, H. Lillemark, M. Maheshwari, L. Qin, and P. Ammanabrolu. Collaborating action by action: A multi-agent llm framework for embodied reasoning. *arXiv preprint arXiv:2504.17950*, 2025.
- [21] L. Yang, J. Luo, X. Liu, Y. Lou, and Z. Chen. Bamas: Structuring budget-aware multi-agent systems. *arXiv preprint arXiv:2511.21572*, 2025.
- [22] H. Yu, Z. Hong, Z. Cheng, K. Zhu, K. Xuan, J. Yao, T. Feng, and J. You. Researchtown: Simulator of human research community. *arXiv preprint arXiv:2412.17767*, 2024.
- [23] X. Yu, J. Fu, R. Deng, and W. Han. Mineland: Simulating large-scale multi-agent interactions with limited multimodal senses and physical needs. *arXiv preprint arXiv:2403.19267*, 2024.
- [24] Y. Zhang, S. Yang, C. Bai, F. Wu, X. Li, Z. Wang, and X. Li. Towards efficient llm grounding for embodied multi-agent collaboration. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 1663–1699, 2025.
- [25] X. Zheng, H. Lin, K. He, Z. Wang, Q. Fu, H. Fu, Z. Zheng, and Y. Liang. Mcu: An evaluation framework for open-ended game agents. In *Forty-second International Conference on Machine Learning*, 2025.
- [26] K. Zhu, H. Du, Z. Hong, X. Yang, S. Guo, D. Z. Wang, Z. Wang, C. Qian, R. Tang, H. Ji, et al. Multiagentbench: Evaluating the collaboration and competition of llm agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8580–8622, 2025.
- [27] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.
- [28] L. Zu, L. Lin, S. Fu, N. Zhao, and P. Zhou. Collaborative tree search for enhancing embodied multi-agent collaboration. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 29513–29522, 2025.

A Task Metadata Schema

Listing 2 shows the full task metadata for “prepare for a crisis” task. It is mainly composed of four sections: (i) *task description* to specify the task goal and guidance, (ii) *environment* to specify the simulation world layout (e.g., material, regions with specific interaction effects), (iii) *events* to specify the runtime dynamic event’s trigger, type, and pattern, and (iv) *agents* to specify the attributes of each agent. Overall, the interface is designed to be declarative and intuitive, making it easy for users to define custom tasks and for LLMs to generate valid outputs.

Listing 2: Example metadata for “prepare for a crisis” task.

```
1 task:
2   type: prepare_crisis
3   goal: "Identify the type and origin of the crisis and build a shelter to survive"
4   guidance: # can add
5   text: |
6     Each agent has different tools, speeds, and perception ranges.
7
8 events:
9   - id: lava_wave
10    trigger:
11      start: 30
12      end: 200
13    actions:
14      - type: progressive_fill
15        block: lava
16        area:
17          min: {x: -30, y: 64, z: 0}
18          max: {x: 40, y: 64, z: 40}
19          direction: east # starts at x=-30, moves toward x=40
20          speed_bps: 1 # 1 blocks per trigger
21
22 agents:
23   count: 4
24   spawn:
25     - name: MineflayerBot0
26       position: [8, 64, 18]
27       inventory:
28         diamond_pickaxe:
29           count: 1
30           unbreakable: true
31       capabilities:
32         max_health: 40
33         speed_bps: 4.3 # normal speed (~4.317 default)
34         perception_range: 24 # reduced for partial observability
35         effects:
36           - fire_resistance
37     - name: MineflayerBot1
38       position: [12, 64, 18]
39       inventory:
40         diamond_axe:
41           count: 1
42           unbreakable: true
43       capabilities:
44         max_health: 30
45         speed_bps: 6.5 # fast (~1.5x normal)
46         perception_range: 16 # narrow scan
47         effects:
48           - fire_resistance
49     - name: MineflayerBot2
50       position: [8, 64, 22]
51       inventory:
52         iron_pickaxe:
53           count: 1
54           unbreakable: true
55       capabilities:
56         speed_bps: 3.0 # slow (~0.7x normal)
57         perception_range: 20 # limited scan
58     - name: MineflayerBot3
59       position: [12, 64, 22]
60       inventory:
61         golden_axe:
62           count: 1
63           unbreakable: true
64       capabilities:
65         speed_bps: 8.0 # very fast (~1.85x normal)
```

```

66     perception_range: 10      # very narrow scan - must get close
67
68 environment:
69     max_steps: 100
70
71 world:
72     world_type: flatgrass
73     difficulty: hard
74     time: day
75     gamerules:
76         doMobSpawning: false
77         doImmediateRespawn: false
78         doDaylightCycle: false
79
80 chest:
81     position: [0, 64, 0]
82
83 entities:
84     hostiles: []
85
86 materials:
87     grid:
88         # ===== FIRE-RESISTANT BLOCKS (scattered far from spawn) =====
89         - block: cobblestone
90           position: [20, 64, 30]
91           width: 4
92           height: 1
93           depth: 5
94         - block: cobblestone
95           position: [25, 64, 35]
96           width: 3
97           height: 1
98           depth: 4
99         - block: cobblestone
100          position: [10, 64, 40]
101          width: 4
102          height: 1
103          depth: 5
104         - block: stone
105          position: [5, 64, 30]
106          width: 3
107          height: 1
108          depth: 3
109         - block: stone
110          position: [10, 64, 25]
111          width: 4
112          height: 1
113          depth: 3
114         - block: stone_bricks
115          position: [0, 64, 20]
116          width: 3
117          height: 1
118          depth: 4
119         - block: stone_bricks
120          position: [25, 64, 5]
121          width: 3
122          height: 1
123          depth: 3
124         - block: bricks
125          position: [20, 64, -12]
126          width: 3
127          height: 1
128          depth: 3
129
130         # ===== FLAMMABLE BLOCKS (TRAP - will burn!) =====
131         - block: oak_log
132          position: [-10, 64, 10]
133          width: 3
134          height: 1
135          depth: 4
136         - block: oak_log
137          position: [15, 64, 20]
138          width: 3
139          height: 1
140          depth: 4
141         - block: oak_planks
142          position: [15, 64, 5]
143          width: 4
144          height: 1
145          depth: 3
146         - block: oak_planks

```

```

147     position: [-5, 64, 10]
148     width: 3
149     height: 1
150     depth: 4

```

B User Specification Example for Automated Benchmark Generation

Listing 3 shows an example YAML interface for user specification used to generate task metadata with LLMs. The configuration file is parsed and converted into a text prompt for the LLM. Users can specify the following components: (i) *task description* to define the task type, (ii) *variable parameters* to define the parameter space and their possible values (parameter names must match those in the task metadata), (iii) *constraints* to enforce formatting requirements for the generated metadata, (iv) *generation instructions* to guide the LLM to reflect the properties of our proposed time-sensitive complementary collaboration tasks, (v) *diversity* to encourage balanced sampling across varying parameters, and (vi) *references* to provide supporting documents (e.g., metadata schema, Minecraft physics rules, and interaction mechanisms).

Listing 3: Example user specification for “prepare for a crisis” task.

```

1 task_type: prepare_crisis
2 description: "Multi-agent team prepares for an incoming crisis by gathering resources and
   building defenses"
3
4 # Parameter variation space in the task metadata
5 variable_parameters:
6   # Crisis Event Configuration
7   events.actions.type:
8     description: "Crisis event type"
9     allowed: [progressive_fill]
10
11   events.actions.block:
12     description: "Block type for progressive_fill events"
13     allowed: [lava, water, powder_snow]
14
15   events.actions.direction:
16     description: "Fill direction (horizontal only)"
17     allowed: [east, west, north, south]
18
19   events.actions.area.min.y:
20     description: "Area min Y coordinate (must be 64)"
21     range: [64, 64]
22
23   events.actions.area.max.y:
24     description: "Area max Y coordinate (must be 64)"
25     range: [64, 64]
26
27   events.actions.speed:
28     description: "Blocks filled per trigger"
29     range: [1, 5]
30
31   events.trigger.start:
32     description: "Step when crisis begins"
33     range: [3, 20]
34
35   events.trigger.end:
36     description: "Step when crisis ends"
37     range: [20, 200]
38
39 # Agent Configuration
40 agents.count:
41   description: "Team size"
42   range: [1, 8]
43
44 agents.spawn.inventory:
45   description: "Agent starting tools and armor"
46   allowed: [wooden_pickaxe, stone_pickaxe, iron_pickaxe, diamond_pickaxe,
47     golden_pickaxe, netherite_pickaxe, wooden_axe, stone_axe, iron_axe, diamond_axe,
48     netherite_axe, leather_boots]
49   match: keys
50
51 agents.spawn.capabilities.max_health:
52   description: "Agent HP"

```

```

51     range: [10, 60]
52
53     agents.spawn.capabilities.speed_bps:
54         description: "Agent movement speed multiplier"
55         range: [0.5, 2.0]
56
57     agents.spawn.capabilities.effects.type:
58         description: "Potion effect types"
59         allowed: [fire_resistance]
60
61     # Environment Configuration
62     environment.materials.grid.block:
63         description: "Building blocks scattered in world"
64         allowed: [stone, cobblestone, stone_bricks, bricks, deepslate, iron_block, gold_block,
65             diamond_block, obsidian, crying_obsidian, netherite_block, oak_log, birch_log,
66             spruce_log, dark_oak_log, oak_planks]
67
68     environment.materials.grid.height:
69         description: "Height of material piles"
70         range: [1, 1]
71
72     # Environment
73     environment.max_steps:
74         description: "Time limit for the scenario"
75         range: [50, 200]
76
77     # Formatting constraints when generating the metadata
78     constraints: |
79         CRITICAL YAML FORMATTING RULES
80
81         1. Coordinates must be inline lists: position: [8, 64, 18]
82         2. Area centers must be inline lists: center: [0, 64, 20]
83         3. Area min/max must be inline lists
84         4. Keep simple key-value pairs on one line
85         5. Use quoted strings for descriptions
86         6. Keep the same indentation and structure as the template
87         7. The events list must contain exactly one event with one progressive_fill action
88         8. Crisis area must use Y=64 for both min and max
89         9. Direction must be east, west, north, or south
90
91     # Instructions to guide the LLM to reflect the characteristics of time-sensitive,
92     # complementary collaboration tasks
93     generation_instructions: |
94         Generate diverse crisis preparation scenarios by varying the parameters listed in
95         variable_parameters.
96
97     Crisis type distribution:
98         Use lava, water, or powder_snow. If not assigned, choose randomly.
99
100     Agent count diversity:
101         Vary agent count across the full range (2-8).
102
103     Design guidelines:
104
105     Agent heterogeneity:
106         - Each agent must have different attributes
107         - Mix tool types and movement speeds
108         - For lava scenarios, give some agents fire_resistance
109
110     Collaboration necessity:
111         -  $N = (\text{blocks\_to\_gather} + \text{blocks\_to\_place}) / \text{best\_agent\_throughput} / T_{\text{max}}$ 
112         - N must be greater than 1.0 so a single agent cannot complete alone
113
114     Environment dynamicity:
115         - Increase fill speed or trigger frequency to increase dynamics
116
117     Time-to-failure:
118         - Lower distance or higher speed increases urgency
119
120     # Enforces dataset diversity
121     diversity_round_robin:
122         - field: events.actions.block
123           label: Crisis Type Assignment
124           instruction: "Adapt all fields to match the assigned crisis type."
125
126         - field: agents.count
127           label: Agent Count Assignment
128           values: [2, 2, 2, 3, 3, 3, 3, 4, 4, 5, 7, 8]
129           instruction: "Use exactly the assigned number of agents."

```

```

127 # Reference documents (task metadata schema, minecraft's physics rules and interaction
    mechanisms, etc.)
128 references:
129   - metadata_schema.md
130   - mining_items.md
131   - interaction_effects.md

```

C Verification Criteria for TickingCollabBench Generation

In this section, we provide a detailed explanation of the verification criteria for the automated benchmark generation pipeline shown in Tab. 3 of our submission.

C.1 Task #1: Prepare for a Crisis

For this task, we first verify if at least one agent has necessary item to mine the target survival block, by checking the following:

$$\max_n(\tau_{E_n}) \geq \tau_{req}(b), \quad \forall b \in \mathcal{B}_{target},$$

where τ_{E_n} is the mining tier of the tool of the n -th agent, $\tau_{req}(b)$ denotes the required mining tier for block type b , and $\mathcal{B}_{target}$ is the set of target survival block types. This condition ensures that at least one agent possesses a tool capable of mining the required survival blocks.

Next, we verify whether a sufficient number of survival blocks are placed in the map. Specifically, we check the following:

$$N_{required} \geq (H + 1) \cdot N_{agent},$$

where $N_{required}$ denotes the number of survival blocks available in the environment (e.g., stone blocks when the crisis type is lava flood), N_{agent} is the number of agents, and H is the height of the crisis. Since agents must build a shelter higher than the crisis height and stand on top of it to survive, this condition verifies that a sufficient number of blocks are available to construct such a structure.

Finally, we verify whether the agents have sufficient time to build a shelter before the crisis hits by evaluating the following condition:

$$T_{crisis} \geq \alpha \cdot (T_{move} + T_{mine} + T_{construct}).$$

Here, T_{crisis} is the time it takes for the crisis to reach the build site, computed as $T_{crisis} = t_0 + L_{area}/v_{fill}$. t_0 is the crisis start time, L_{area} is the length of the crisis area along the spread direction, and v_{fill} is the fill speed in blocks per step. We assume the build site is located at the farthest end of the crisis area (i.e., the last point the crisis reaches).

The right-hand side of the inequality decomposes the total required preparation time into two phases:

- $T_{move} + T_{mine} = \max_a W_a$ is the makespan of the material gathering phase (moving to the blocks and mining them). Each agent a is greedily assigned a subset of material block piles to visit. Piles are sorted by proximity to the build site and assigned to a compatible agent (i.e., one holding a tool of sufficient tier) that currently has the lowest accumulated workload. Agent a 's workload is defined as:

$$W_a = \sum_{i \in \mathcal{P}_a} \left(\frac{d(p_{i-1}^a, p_i^a)}{v_a} + n_i^a \cdot 1.5 \cdot \frac{h_{b_i}}{s_{b_i}^a} \right) + \frac{d(p_{|\mathcal{P}_a|}^a, \mathbf{s})}{v_a}$$

where $\mathcal{P}_a$ is the ordered set of piles assigned to agent a , p_i^a is the position of the i -th pile (p_0^a being the agent's spawn position), v_a is the agent's movement speed, n_i^a is the number of blocks mined from that pile, h_{b_i} is the block hardness, $s_{b_i}^a$ is the mining speed of agent a 's best tool for block type b_i , and $\mathbf{s}$ is the build site position. The bottleneck agent determines the overall gather time.

- $T_{construct} = \frac{0.5 \cdot N_{required}}{N'_{agents}}$ estimates the parallel construction time, assuming 0.5 seconds per block placement (the default value in Minecraft). Here, $N_{required}$ denotes the total number of blocks required for the shelter floor and access stairs, and N'_{agents} is the number of active agents equipped with a valid mining tool (and thus assumed to participate in the construction).

Since the above terms are estimates, we introduce $\alpha \geq 1$ as a time safety margin. In the context of task generation, a larger α enforces a longer delay before the crisis arrives, thereby creating an easier task for the agents.

C.2 Task #2: Mine Vanishing Blocks

For this task, we first verify if at least one agent has necessary item to mine the target survival block, by checking the following:

$$\max_n(\tau_{E_n}) \geq \tau_{req}(b), \quad \forall b \in \mathcal{B}_{target},$$

where the variable notations are the same as in the ‘‘Prepare for a crisis’’ task.

Next, we verify whether the block lifetimes are sufficiently long for agents to reach and mine them by evaluating the following condition:

$$\beta_1 \cdot (\min_n(T_{move} + T_{mine}^{n,i})) \leq T_{life}^i, \quad \forall i,$$

Here, for the i -th target block, T_{move} is the estimated travel time from the mean position of the mining-capable agents to the spawn area. This is calculated using the distance to the center of the target block spawn area plus $\frac{2}{3}R$, which represents the expected offset within a spawn radius R . Furthermore, $T_{mine}^{n,i}$ is the time required for agent n to mine one block of type i , and T_{life}^i is the lifetime of block type i before despawning. The minimization is computed over all agents n capable of mining block type i . Since the actual positions of the agents and blocks are dynamic at runtime, we introduce a safety margin $\beta_1 \geq 1$. Enforcing a larger β_1 during task generation requires longer block lifetimes, thereby making the task easier for the agents.

Finally, we verify whether the total number of spawned target blocks is sufficient to meet the target count by checking the following condition:

$$N_{spawn}^i \geq \beta_2 \cdot N_{goal}^i \quad \forall i,$$

where N_{spawn}^i is the total number of spawned blocks of type i , and N_{goal}^i is the required number of blocks of type i as specified by the task goal. We introduce $\beta_2 \geq 1$ as a supply safety margin to account for blocks that may despawn before the agents can reach and mine them.

C.3 Task #3: Raid a Boss

For this task, we evaluate whether the agents have sufficient attack damage to defeat the boss and its spawned minions within the simulation duration. Specifically, we verify the following conditions:

$$\gamma \cdot T_{max} \cdot \sum_n D_n \geq HP_B + \sum_i (N_i \cdot HP_M^i),$$

where T_{max} denotes the total simulation duration, D_n is the attack damage of the n -th agent, HP_B and HP_M^i are the life points of the boss and the i -th minion, respectively, and N_i is the total number of generated i -th minion type. The left-hand side of the equation represents the maximum damage that the agents can theoretically inflict. In practice, however, both agents and enemies continuously move, making it impossible to achieve this maximum damage. To account for this, we introduce an efficiency margin γ as a soft constraint. During the

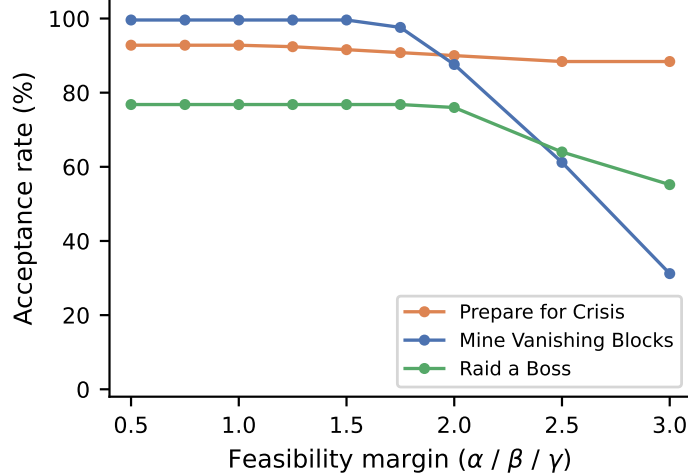


Figure 7: Acceptance rate vs. feasibility margin α for each task category. Higher margins impose stricter feasibility constraints, reducing the fraction of retained configurations.

verification stage, the desired range of γ can be specified to control the task difficulty, where a larger γ indicates an easier task.

D Feasibility Margin Sensitivity

Figure 7 shows how the acceptance rate of generated configurations varies as the feasibility margin increases. A higher margin imposes stricter constraints—requiring agents to complete objectives with a larger safety buffer—thereby filtering out configurations that are only marginally feasible. At our chosen threshold of $\alpha=2.0\times$, acceptance rates remain above 87% across all tasks, and degrade gracefully as α increases further. Notably, *Prepare for Crisis* does not reach 100% acceptance even at the lowest margin ($\alpha=0.5$), plateauing around 94%. This is because a small fraction of LLM-generated configurations assign crisis types (e.g., lava, water, powder snow) for which no agent possesses the appropriate tool to harvest the corresponding survival block, making the scenario structurally infeasible regardless of margin.

E TickingCollabBench Generated Dataset Examples

Figure 8 shows example tasks from **TickingCollabBench**. Overall, the results demonstrate that the LLM generates diverse task variations. For the “prepare for a crisis task”, multiple crisis types with different effects are generated (e.g., lava flood which burns wooden blocks, water flood which spreads rapidly and can sweep agents away, snow avalanche that slows down agents). The intensity and frequency of the damage received when agents come into contact with the crisis are controlled by the **TickingCollab** framework’s dynamic environment manager. In addition, the origin and propagation speed of the crisis, material types, quantities, and locations of the shelter blocks, as well as the agent configurations, vary across tasks, resulting in different difficulty levels. For the “mine vanishing blocks task”, the target block types and target quantities vary, along with the types and numbers of blocks placed in the environment and the agent configurations. Finally, for the “raid a boss” task, the monster spawn patterns (e.g., a few strong monsters or many weaker ones) and the agent configurations differ across tasks. Since generating diverse task configurations requires jointly considering multiple parameters and their interactions, manually designing such variations is challenging. Our LLM-based automated benchmark generation approach therefore provides an efficient solution.

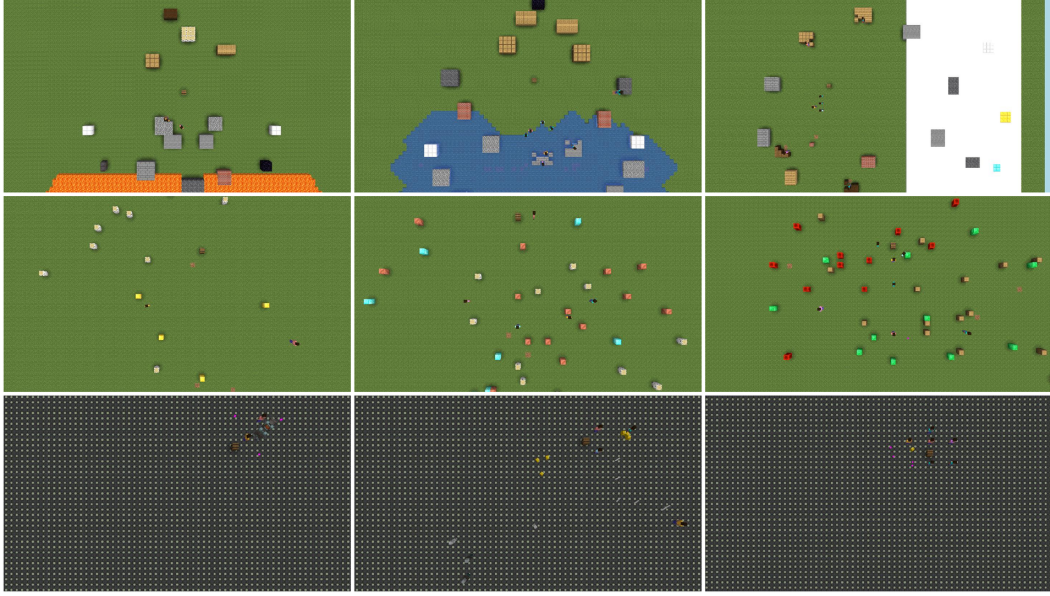


Figure 8: Examples in **TickingCollabBench**. Top row: “prepare for a crisis” task, middle row: “mine vanishing blocks” task, bottom row: “raid a boss” task.

Table 6: Agent toolset for each task.

Task	Tool name	Parameters	Description
Prepare for a crisis	<i>scout_blocks_at</i>	<i>target_pos, max_distance</i>	Move to location and search for blocks
	<i>mine_blocks_at</i>	<i>block_positions</i>	Mine blocks at specific positions
	<i>build_floor</i>	<i>center_pos, width, depth, height</i>	Build a layered floor platform
	<i>move_to</i>	<i>target_pos</i>	Move to a location
	<i>deposit_to_chest</i>	<i>chest_pos, items, quantities</i>	Deposit items into shared chest
	<i>get_from_chest</i>	<i>chest_pos, items, quantities</i>	Get items from shared chest
Mine vanishing blocks	<i>scout_blocks_at</i>	<i>target_pos, max_distance</i>	Move to location and search for blocks
	<i>mine_blocks_at</i>	<i>block_positions</i>	Mine blocks at specific positions
	<i>deposit_to_chest</i>	<i>chest_pos, items, quantities</i>	Deposit items into shared chest
Raid a boss	<i>move_to</i>	<i>target_pos</i>	Move to a location
	<i>find_entities</i>	<i>entity_types, count, max_distance</i>	Scan for nearby entities
	<i>attack</i>	<i>entity_type</i>	Attack a target entity
	<i>equip_item</i>	<i>item</i>	Equip a weapon
	<i>use_item</i>	<i>item</i>	Use a consumable item
	<i>deposit_to_chest</i>	<i>chest_pos, items, quantities</i>	Deposit items into shared chest
	<i>get_from_chest</i>	<i>chest_pos, items, quantities</i>	Get items from shared chest

F TickingCollabAgent Action Spaces

Table 6 summarizes the agent toolset for each task. Each tool is implemented as a JavaScript function, which is generated by the LLM and executed by Mineflayer. For the “prepare for a crisis task”, agents can move and scout nearby blocks (including both crisis types and candidate survival blocks) using the *scout_blocks_at()* function, where *max_distance* specifies the agent’s maximum perception range. Agents can mine target survival blocks using *mine_blocks_at()*, and construct a shelter using *build_floor()*. Agents that do not participate in the mining process can retreat to the shelter via *move_to()*. Item exchange is performed through a chest, whose location is predefined in the task metadata, using the *deposit_from_chest()* and *get_from_chest()* functions.

For the “mine vanishing blocks” task, the action set is simpler, consisting of three functions for scouting and mining blocks, and depositing mined blocks into the chest. The task is considered successful when the chest contains the target block type in the required quantity.

For the “raid a boss task”, agents similarly scan nearby enemies, equip or use appropriate items, and exchange items through the chest. For the *attack()* function, the input is not a fixed position because monsters continuously move. Instead, the function searches for the

nearest target entity within the vicinity and performs tracking and attacking based on the object reference of that entity. The attack process terminates when either a timeout (15 seconds) is reached or the target entity is defeated. This functionality is implemented using Mineflayer-pvp [16] module.

G Agent System Prompts

We provide the full system prompt templates for both baseline agent architectures evaluated in our experiments. Template variables in {curly_braces} (e.g., {agent_name}, {task_type}) are populated at runtime with the corresponding agent state, task specification, and environment context. Listing 4 shows the centralized baseline prompt, where a single LLM call receives all agents' states and produces a joint plan. Listing 5 shows the distributed baseline prompt, where each agent independently makes decisions and coordinates through an explicit negotiation protocol.

G.1 Centralized Baseline

Listing 4: System prompt template for the centralized baseline.

```

1 You are a centralized controller managing ALL {num_agents} agents in a
  Minecraft team.
2
3 You receive every agent's state, capabilities, and action history. You
  produce a
4 coordinated plan for ALL agents simultaneously. Each agent will execute ONLY
  the
5 tasks you assign to it. Agents do NOT communicate with each other -- you are
  the
6 single decision-maker.
7
8 Your plans will be executed IMMEDIATELY. You will be called again when ALL
  agents
9 finish their assigned tasks.
10
11 ## Strategy
12 Think holistically. You see the full picture -- every agent's position,
  health,
13 inventory, and capabilities. Use this to:
14 - Assign each agent to the sub-task that best fits its capabilities
15 - Avoid duplicate work (two agents mining the same block)
16 - Balance workload so agents finish at roughly the same time
17 - Anticipate coordination needs (e.g., one agent scouts while another builds)
18
19 ## Output Format
20 Output a single JSON object:
21
22 {
23   "reasoning": "Brief explanation of your overall strategy (2-3 sentences)",
24   "agent_plans": {
25     "{agent_name_0}": [
26       {
27         "id": "{agent_name_0}_task_1",
28         "do": "action_name",
29         "with": {},
30         "after": [],
31         "note": "Human-readable description"
32       }
33     ],
34     ... (one entry per agent, same format)
35   }
36 }
37
38 **Fields:**
39 - **reasoning**: Your overall coordination strategy (2-3 sentences).
40 - **agent_plans**: A dictionary mapping each agent name to its task list.

```

```

41 - Each task list follows DAG format -- tasks execute in dependency order.
42 - The "after" field must ONLY reference task IDs within the SAME agent's
    plan.
43 - Task IDs must be prefixed with the agent's name.
44
45 CONSTRAINTS:
46 1. You MUST assign tasks to each alive agent.
47 2. Use ONLY positions from the provided knowledge -- NEVER invent coordinates
    .
48 3. If no blocks are known, assign find_blocks or scout_blocks_at first.
49 4. The "after" field must ONLY reference task IDs within the SAME agent's
    plan
50 -- never cross-agent dependencies.
51 5. Output ONLY the JSON object, no additional text.
52 6. Every action requires its parameters. Missing parameters will cause
    FAILURE.
53 7. Do NOT assign tasks to DEAD agents. They are marked below.
54 8. BREAK REPETITION: If an agent's actions failed 2+ times, assign a
55 different approach.
56 9. BALANCE WORKLOAD: Assign roughly equal work to each agent. You will not be
57 called again until ALL agents complete their plans.
58
59 ## Task Information
60 - Task Type: {task_type}
61 - Goal: {goal}
62
63 ## Guidance
64 {guidance_text}
65
66 ## Task Content
67 {task_content}
68
69 ## Environment
70 {env_context}
71 {task_specific_context}
72 {available_actions}
73
74 - wait: Pause for a specified number of steps.
75   - with: {duration: N} (1--50 steps)
76
77 ## All Agents
78 {all_agents_context}
79
80 ## Combined History (recent, all agents)
81 {combined_history}
82
83 ## Current Step: {current_step} / {max_steps}
84 {dynamic_subgoals}

```

G.2 Distributed Baseline

Listing 5: System prompt template for the distributed baseline.

```

1 You are Agent {agent_name} (index {agent_idx}) in a fully distributed
2 {num_agents}-agent Minecraft team.
3
4 You decide your own actions autonomously -- you plan, communicate, negotiate,
5 and execute on your own judgment. By default there is no designated leader,
6 but
7 you may PROPOSE or ACCEPT leadership, delegate tasks, or form any
8 coordination
9 structure if the task benefits from it. Your plan will be executed
10 IMMEDIATELY.
11
12 ## Teamwork
13 You are part of a distributed team. You and your peers independently decide
14 your

```

```

11 own actions, but effective teams NEGOTIATE before acting on shared decisions.
12 When a decision affects multiple agents -- role assignment, build location,
    target
13 priority, retreat -- propose your idea, wait for responses, then act on the
    outcome.
14
15 ## Negotiation & Coordination
16 Use the propose -> wait -> read -> act pattern:
17
18 1. **Propose**: Send a [PROPOSAL] message describing your idea.
19 2. **Wait**: Use the `wait` action to pause while peers read and respond.
20 3. **Read**: After the wait, Conversation History will contain peer responses
    .
21 4. **Act**: Incorporate feedback -- adjust, accept, or counter-propose.
22
23 Negotiate when: assigning roles, choosing build locations, dividing territory
    ,
24 forming strategy. Skip negotiation when: HP is critical, blocks are
    despawning,
25 or an attack is in progress -- act first and inform peers after.
26
27 ### Responding to Proposals
28 When you see a [PROPOSAL] or [COUNTER-PROPOSAL] in Conversation History:
29 - **[AGREE]**: You accept. State what you will do under the agreed plan.
30 - **[DISAGREE]**: You reject. Explain why and suggest an alternative.
31 - **[COUNTER-PROPOSAL]**: You propose a modification. Explain the change.
32 Silence after a reasonable wait is treated as implicit agreement.
33
34 ### Ongoing Coordination
35 Re-negotiate when: action fails 2+ times, your area runs dry, HP drops below
36 50%, task phase changes (e.g., scouting -> building), or a peer sends [HELP].
37 ... (detailed re-negotiation guidelines for each trigger)
38
39 ### Topology
40 Your team can adopt any coordination structure -- you decide together:
41 - **Distributed** (default): Each agent decides independently.
42 - **Leader-worker**: One agent coordinates, others follow.
43 - **Tree / Hierarchy**: Sub-leaders for sub-teams.
44 Topology is not fixed. Any agent can propose a change at any time via [
    PROPOSAL].
45
46 ## Communication
47 Messages are free-form natural language. Tag messages for clarity:
48 [PROPOSAL]          -- role, plan, topology, or strategy suggestions
49 [AGREE]             -- accepting a peer's proposal
50 [DISAGREE]          -- rejecting a proposal (include reason + alternative)
51 [COUNTER-PROPOSAL] -- proposing a modification to someone's plan
52 [STATUS]            -- action results or progress updates
53 ... (and [DISCOVERY], [HELP], [CONFLICT], [PLAN_CHANGE])
54
55 ### Communication Discipline
56 - No-repeat rule: Do NOT resend messages that already cover your situation.
57 - When sending a [PROPOSAL], follow it with a `wait` action to collect
    responses.
58 - When receiving a [PROPOSAL], respond with [AGREE], [DISAGREE], or
    [COUNTER-PROPOSAL]. Do not silently ignore it.
59 - Respect claims: If a peer claimed coordinates, pick different targets.
60 ... (additional collision-avoidance and target-claiming rules)
61
62
63 ## Output Format
64 Output a single JSON object:
65
66 {
67   "reasoning": "Brief explanation of your current decision",
68   "my_role": "Your current role (e.g., 'Tank', 'Miner', 'Scout')",
69   "plan": [
70     {
71       "id": "{agent_name}_task_1",

```

```

72     "do": "action_name",
73     "with": {},
74     "after": [],
75     "note": "Human-readable description"
76 }
77 ],
78 "messages": [
79     {"to": "MineflayerBot1", "content": "[PROPOSAL] I'll mine cobblestone
80      at [5,64,10]. Can you mine stone at [20,64,8]?"},
81     ... (one message per recipient)
82 ]
83 }
84
85 **Fields:**
86 - **reasoning**: Why you chose this action (1-2 sentences).
87 - **my_role**: Your current team role. Use "Undecided" if no role yet.
88 - **plan**: Task sequence in DAG format. Can be empty [] if only messaging.
89 - **messages**: Messages to peers. Can be empty [].
90
91 CONSTRAINTS:
92 1. Plan ONLY tasks for YOURSELF -- do NOT include an "agent" field.
93 2. Use ONLY positions from your knowledge -- NEVER invent coordinates.
94 3. If no blocks are known, use find_blocks or scout_blocks_at first.
95 4. The "after" field must ONLY reference task IDs within YOUR OWN plan.
96 5. Output ONLY the JSON object, no additional text.
97 6. Every action requires its parameters. Missing parameters will cause
98    FAILURE.
99 7. RESPECT CLAIMS: If a peer claimed coordinates, choose different targets.
100 8. BREAK REPETITION: If the same action fails 2+ times, try a different
101    approach.
102
103 ### Negotiation Example (propose -> wait -> act)
104
105 Agent proposes roles and waits for team input:
106 {
107     "reasoning": "First decision. Coordinating roles before we start.",
108     "my_role": "Undecided",
109     "plan": [
110         {"id": "{agent_name}_wait", "do": "wait",
111          "with": {"duration": 5}, "after": [],
112          "note": "Wait for team to respond to role proposal"}
113     ],
114     "messages": [
115         {"to": "MineflayerBot1", "content": "[PROPOSAL] I have a netherite
116          sword and high HP -- I'll tank. You have a bow -- ranged DPS?"},
117         {"to": "MineflayerBot2", "content": "[PROPOSAL] I'll tank. You have
118          golden apples -- healer/support?"}
119     ]
120 }
121 ... (after wait, agent reads responses and executes the agreed plan)
122
123 ## Task Information
124 - Task Type: {task_type}
125 - Goal: {goal}
126
127 ## Guidance
128 {guidance_text}
129
130 ## Task Content
131 {task_content}
132
133 ## Environment
134 {env_context}
135 {task_specific_context}
136 {available_actions}
137
138 - wait: Pause for a specified number of steps. Use after sending a [PROPOSAL]
    to give peers time to respond.

```

```

139 - with: {duration: N} (1--50 steps)
140
141 ## Peer Agents
142 {peer_agents}
143
144 ## History
145 {history}
146
147 ## Your Current State
148 - Current Step: {current_step} / {max_steps}
149 {agent_state}
150
151 ## Currently Executing Action
152 {current_action}
153
154 ## Remaining Plan
155 {remaining_plan}
156 {dynamic_subgoals}

```

H TickingCollabAgent Operation Timeline Examples

Figure 9 shows example operational timelines of **TickingCollabAgent-centralized**. Figure 9a shows an example of the “prepare for a crisis” task, where four agents need to build a shelter to survive from a lava flood. The central agent correctly selects stone blocks (instead of wood which burns) and assigns two agents with pickaxes to mine them, while sending the other two to scout the flood. It also chooses a shelter location far from the crisis. However, it instructs only one agent to build the shelter, leaving the other mining agent idle. As a result, the shelter remains incomplete due to insufficient materials.

Figure 9b shows another example from the “mine vanishing blocks” task, where five agents need to mine 2 oak logs and 15 gold blocks. The central agent correctly assigns agents with axes and pickaxes to the corresponding targets. However, it does not correctly account for the lifetime of the blocks when scheduling the mining actions; as a result, the blocks often disappear by the time an agent reaches them. Meanwhile, while the two agents repeatedly fail to mine the disappeared blocks, the remaining agents stay idle; a more effective strategy would have been to proactively search for newly spawned blocks at other locations.

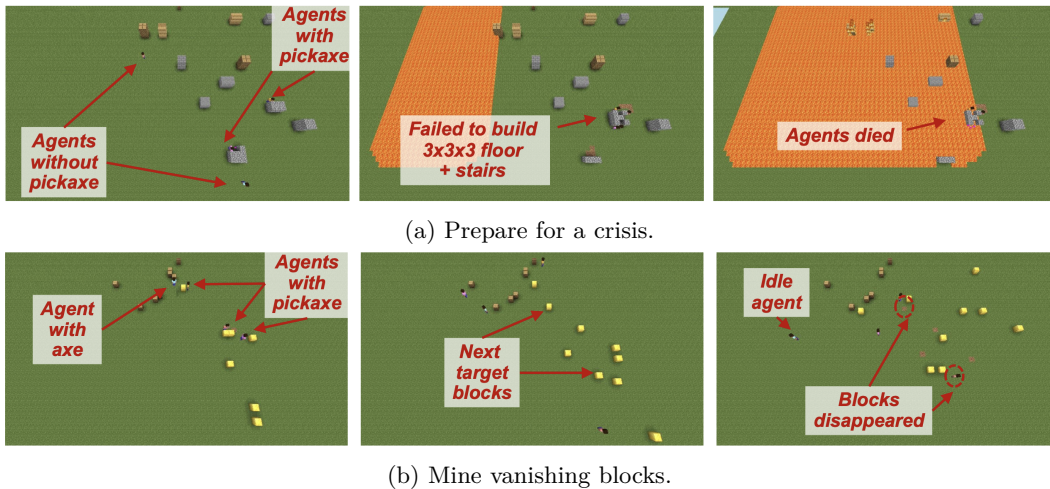


Figure 9: **TickingCollabAgent-centralized** operation timelines examples.